Homework 01

Haoran Xu

2026-09-06

Question 1 (Least squares and matrix calculations)

Solution

Part a. *(dimensions and rank of $\mathbf{X}$; QR least-squares fit; comparison with the generating coefficients.)* QR solves $\mathbf{R}\widehat{\beta} = \mathbf{Q}^{\mathsf{T}}\mathbf{y}$ by back substitution, never forming $\mathbf{X}^{\mathsf{T}}\mathbf{X}$ or its inverse.

```
set.seed(43201)
n <- 100
p <- 5
Xmat <- matrix(rnorm(n * p), nrow = n, ncol = p)
colnames(Xmat) <- paste0("x", 1:p)
epsilon <- rnorm(n, mean = 0, sd = 0.7)

X <- cbind(intercept = 1, Xmat)
beta_true <- c(3, 1.4, -0.9, 0.7, 0, 1.1)
y <- as.vector(X %*% beta_true + epsilon)

dim(X)
#> [1] 100   6
qr_X <- qr(X)
qr_X$rank
#> [1] 6
```

$\mathbf{X}$ is 100×6 with full column rank 6, so the solution is unique.

```
beta_hat <- qr.coef(qr_X, y)

# standard errors from the R factor (chol2inv avoids inverting t(X) %*% X)
XtX_inv <- chol2inv(qr.R(qr_X))
rss0 <- sum((y - X %*% beta_hat)^2)
se_hat <- sqrt(diag(XtX_inv) * rss0 / (n - ncol(X)))

cmp <- data.frame(
  truth = beta_true,
  estimate = as.vector(beta_hat),
  difference = as.vector(beta_hat) - beta_true,
  z = (as.vector(beta_hat) - beta_true) / se_hat
)
rownames(cmp) <- colnames(X)
round(cmp, 4)
#>           truth estimate difference      z
#> intercept   3.0   3.0063     0.0063 0.0892
#> x1          1.4   1.4172     0.0172 0.2434
#> x2         -0.9  -0.8883     0.0117 0.1592
#> x3          0.7   0.7953     0.0953 1.1104
```

```
#> x4          0.0    0.0484     0.0484 0.7200
#> x5          1.1    1.1319     0.0319 0.4779

max(abs(beta_hat - lm.fit(X, y)$coefficients))   # agrees with lm.fit
#> [1] 0
```

The largest absolute deviation is 0.0953 and every standardized deviation is under 1.11, well inside ± 2. Note $\hat{\beta}_{x_4} = 0.0484$ against a true 0: with slope standard errors near $\sigma/\sqrt{n} \approx 0.07$, that is what pure noise produces. The discrepancies are $O(n^{-1/2})$ sampling variability, not bias.

Part b. *(fitted values, residual vector, and training RMSE.)*

```
fitted_vals <- as.vector(X %*% beta_hat)
r <- y - fitted_vals

rss <- sum(r^2)
rmse <- sqrt(mean(r^2))
c(RSS = rss, RMSE = rmse, mean_residual = mean(r))
#>           RSS          RMSE mean_residual
#>   4.57476e+01   6.76370e-01   4.10783e-16
```

The training RMSE is 0.6764, slightly *below* $\sigma = 0.7$: the fit absorbs noise across six free coefficients, so $\mathbb{E}[\mathrm{RSS}] = (n-p-1)\sigma^2 < n\sigma^2$, and rescaling by the correct degrees of freedom gives $\sqrt{\mathrm{RSS}/94} = 0.6976$. The residual mean is numerically zero, as it must be with an intercept.

Part c. *($\|\mathbf{X}^\mathsf{T}\mathbf{r}\|_\infty$ and what it says about the residual vector.)*

```
Xtr <- crossprod(X, r)      # t(X) %*% r
max(abs(Xtr))
#> [1] 9.6897e-14
```

$\|\mathbf{X}^\mathsf{T}\mathbf{r}\|_\infty = 9.69 \times 10^{-14}$, zero to machine precision. This is the normal equations restated: the criterion $\|\mathbf{y} - \mathbf{X}\beta\|^2$ has gradient $-2\mathbf{X}^\mathsf{T}(\mathbf{y} - \mathbf{X}\beta)$, so at the minimizer $\mathbf{X}^\mathsf{T}\mathbf{r} = \mathbf{0}$. Geometrically $\hat{\mathbf{y}}$ is the orthogonal projection of $\mathbf{y}$ onto $\mathcal{C}(\mathbf{X})$, so $\mathbf{r} \in \mathcal{C}(\mathbf{X})^\perp$: orthogonal to the intercept column (hence $\sum_i r_i = 0$) and to every predictor column, meaning the residuals hold no remaining *linear* signal in the columns of $\mathbf{X}$. The departure from exact zero is floating-point rounding, of order $\varepsilon_{\mathrm{mach}}\|\mathbf{X}\|\|\mathbf{y}\|$.

Question 2 (Gaussian likelihood for linear regression)

Solution

Part a. *(why maximizing ℓ over β is equivalent to minimizing RSS.)* Writing $\mathrm{RSS}(\beta) = (\mathbf{y} - \mathbf{X}\beta)^\mathsf{T}(\mathbf{y} - \mathbf{X}\beta)$,

$$\ell(\beta, \sigma^2) = \underbrace{-\frac{n}{2}\log(2\pi\sigma^2)}_{\text{free of } \beta} - \frac{1}{2\sigma^2}\mathrm{RSS}(\beta).$$

With $\sigma^2 > 0$ fixed the first term is constant in β and the second is $-c\,\mathrm{RSS}(\beta)$ with $c = 1/(2\sigma^2) > 0$; a strictly decreasing affine transformation reverses optimization, so $\arg\max_\beta \ell = \arg\min_\beta \mathrm{RSS}$ regardless of σ^2. Hence **the Gaussian MLE of β is exactly the OLS estimator**: least squares acquires a likelihood justification under normal errors. Since RSS is convex quadratic with Hessian $2\mathbf{X}^\mathsf{T}\mathbf{X} \succ 0$ at full column rank, it is unique.

Part b. *(MLE of σ^2, and why its denominator differs from the unbiased estimate.)* With $v = \sigma^2$, β fixed and $S = \mathrm{RSS}(\beta)$:

$$\ell = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log v - \frac{S}{2v}, \qquad \frac{\partial \ell}{\partial v} = -\frac{n}{2v} + \frac{S}{2v^2}.$$

Setting this to zero gives $-nv + S = 0$, so $\hat{\sigma}^2_{\mathrm{MLE}} = S/n$. It is a maximum, since at $v = S/n$

$$\frac{\partial^2 \ell}{\partial v^2} = \frac{n}{2v^2} - \frac{S}{v^3} = -\frac{n^3}{2S^2} < 0.$$

```r
sigma2_mle <- rss / n
sigma2_unb <- rss / (n - ncol(X))      # n - p - 1 = 94
c(sigma2_mle = sigma2_mle, sigma2_unbiased = sigma2_unb,
  ratio = sigma2_mle / sigma2_unb, sigma2_true = 0.7^2)
#>      sigma2_mle sigma2_unbiased           ratio     sigma2_true
#>        0.457476        0.486676        0.940000        0.490000
```

So $\hat\sigma^2_{\mathrm{MLE}} = 0.4575$ against $\hat\sigma^2_{\mathrm{unb}} = 0.4867$ and a true 0.49, smaller by the factor $(n-p-1)/n = 94/100$. *Why the denominators differ:* since $\mathbf{r} = (\mathbf{I}-\mathbf{H})\varepsilon$ with $\mathbf{I}-\mathbf{H}$ an idempotent projection of rank $n-p-1$, $\mathbb{E}[\mathrm{RSS}] = (n-p-1)\sigma^2$, so $\mathbb{E}[\hat\sigma^2_{\mathrm{MLE}}] = \frac{n-p-1}{n}\sigma^2$ is biased downward and dividing by $n-p-1$ removes the bias exactly. The intuition is the constraint counting from 1(c): the residuals satisfy the 6 restrictions $\mathbf{X}^\top\mathbf{r} = \mathbf{0}$, so only 94 of the 100 residual coordinates are free. The bias is $O(p/n)$, here 6%.

Part c. *(profile of ℓ along $\mathbf{e}_{x_2}$, and where the maximum falls.)* Because $\mathbf{X}^\top\mathbf{r} = \mathbf{0}$, the cross term $\mathbf{x}_2^\top\mathbf{r}$ vanishes and the profile is an exact downward parabola:

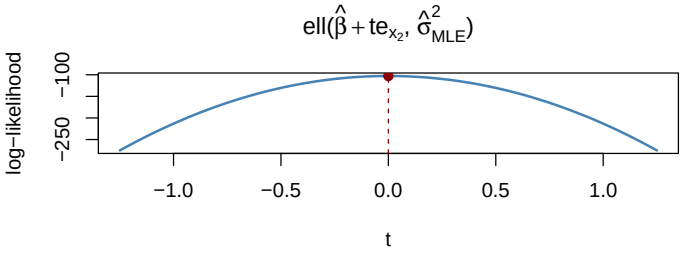
$$\ell\left(\hat\beta + t\mathbf{e}_{x_2}, \hat\sigma^2\right) = \ell(\hat\beta, \hat\sigma^2) - \frac{\|\mathbf{x}_2\|^2}{2\hat\sigma^2}\, t^2.$$

```r
loglik <- function(b, s2) {
  res <- y - X %*% b
  -n / 2 * log(2 * pi * s2) - sum(res^2) / (2 * s2)
}

e_x2 <- c(0, 0, 1, 0, 0, 0)
t_grid <- seq(-1.25, 1.25, length.out = 501)
ll_vals <- sapply(t_grid, function(t) loglik(beta_hat + t * e_x2, sigma2_mle))

t_max <- t_grid[which.max(ll_vals)]
curv <- sum(X[, "x2"]^2) / (2 * sigma2_mle)

plot(t_grid, ll_vals, type = "l", lwd = 2, col = "steelblue",
     xlab = expression(t), ylab = "log-likelihood",
     main = expression(paste(ell, "(", hat(beta) + t * e[x[2]], ", ",
                             hat(sigma)[MLE]^2, ")")))
abline(v = t_max, lty = 2, col = "darkred")
points(t_max, max(ll_vals), pch = 19, col = "darkred")
```




```r
c(t_at_max = t_max, loglik_max = max(ll_vals), curvature = curv)
#>   t_at_max loglik_max  curvature
#>      0.000   -102.792    110.385

# the curve is exactly the parabola predicted above
max(abs(ll_vals - (max(ll_vals) - curv * t_grid^2)))
#> [1] 3.41061e-13
```

The maximum is at $t = 0$ — exactly $\hat\beta$ — with $\ell = -102.792$, and the curve matches $\ell(0) - 110.39\,t^2$ to machine precision. This agrees with least squares because by part a the maximizer over $\mathbb{R}^6$ *is* the least-squares solution, so

it maximizes along any line through it. Concretely, moving along $\mathbf{e}_{x_2}$ perturbs the residual by $-t\mathbf{x}_2$, and since $\mathbf{r} \perp \mathbf{x}_2$ Pythagoras gives $\|\mathbf{r} - t\mathbf{x}_2\|^2 = \|\mathbf{r}\|^2 + t^2\|\mathbf{x}_2\|^2$: any departure strictly increases RSS. The curvature 220.8 is the observed Fisher information for β_{x_2}.

Question 3 (Starting values in nonconvex optimization)

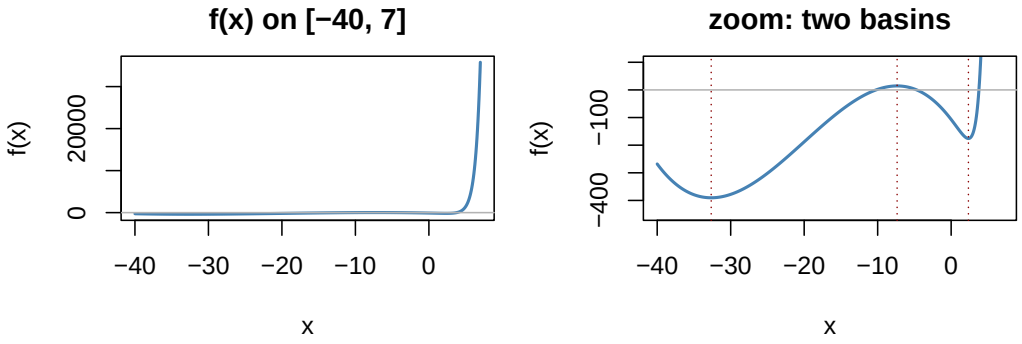
Solution

Part a. *(f on $[-40, 7]$ and the features that affect numerical optimization.)*

```
f <- function(x) exp(1.5 * x) - 3 * (x + 6)^2 - 0.05 * x^3
fp <- function(x) 1.5 * exp(1.5 * x) - 6 * (x + 6) - 0.15 * x^2

xs <- seq(-40, 7, length.out = 4001)
fs <- f(xs)

par(mfrow = c(1, 2), mar = c(4, 4, 2.5, 1))
plot(xs, fs, type = "l", lwd = 2, col = "steelblue",
     xlab = "x", ylab = "f(x)", main = "f(x) on [-40, 7]")
abline(h = 0, col = "grey70")
plot(xs, fs, type = "l", lwd = 2, col = "steelblue",
     ylim = c(-450, 100), xlab = "x", ylab = "f(x)",
     main = "zoom: two basins")
abline(h = 0, col = "grey70")
abline(v = c(-32.649, -7.351, 2.350), lty = 3, col = "darkred")
```




```
par(mfrow = c(1, 1))
```

```
# locate all stationary points by bracketing sign changes of f'
brackets <- which(diff(sign(fp(xs))) != 0)
crit <- sapply(brackets, function(i) {
  uniroot(fp, c(xs[i], xs[i + 1]), tol = 1e-12)$root
})
data.frame(x = crit, f = f(crit),
           type = ifelse((fp(crit + 1e-5) - fp(crit - 1e-5)) / 2e-5 > 0,
                         "local min", "local max"))
#>           x         f      type
#> 1 -32.64911 -390.3858 local min
#> 2  -7.35088   14.3858 local max
#> 3   2.34997 -175.8626 local min
```

Features that matter for optimization:

- **Nonconvex, three stationary points** — local minima near -32.649 and 2.35 split by a local maximum at -7.351, giving **two basins of attraction** of very unequal width; a local descent method stops in whichever basin it starts in.

- **Wildly different scales.** f ranges from about -390 to $35,791$; $\exp(1.5x)$ explodes on the right and both f and f' overflow shortly beyond $x = 7$, so an over-long line-search step can produce non-finite values.
- **A flat left region,** where the exponential is numerically zero and $f \approx -3(x+6)^2 - 0.05x^3$ is shallow, so gradients are small and progress is slow. $f \to +\infty$ at both ends, so a global minimum exists, though the plotted window alone does not prove where it is.

Part b. *(BFGS from $x^{(0)} = -15$ and $x^{(0)} = 0$.)*

```
run_bfgs <- function(start) {
  out <- optim(par = start, fn = f, gr = fp, method = "BFGS")
  data.frame(start = start, x_final = out$par, f_final = out$value,
             abs_grad = abs(fp(out$par)),
             convergence_code = out$convergence,
             converged = out$convergence == 0)
}

bfgs_runs <- rbind(run_bfgs(-15), run_bfgs(0))
bfgs_runs
#>   start   x_final  f_final        abs_grad convergence_code converged
#> 1   -15 -32.64911 -390.386 0.0000000872509                0      TRUE
#> 2     0   2.34997 -175.863 0.0000006556057                0      TRUE
```

Both report `convergence = 0` with $|f'(x)|$ essentially zero, so both reached genuine stationary points — but *different* ones: from -15 to -32.64911 ($f = -390.3858$), and from 0 to 2.34997 ($f = -175.8626$).

Part c. *(why the runs differ, which is lower, and what a global claim would require.)* BFGS is a *local* method: it follows a monotone descent path and guarantees only a stationary point of the basin containing its iterates. Since f is nonconvex, $f'(x) = 0$ has three solutions and the start decides which is reached. The mechanism is the sign of the derivative there: $f'(-15) = 20.25 > 0$ sends the first step **left**, down the wide left basin to -32.649; $f'(0) = -34.5 < 0$ sends it **right**, to 2.35. The starts sit on opposite sides of the interior maximum, and a strictly descending method can never cross that barrier.

The run from $x^{(0)} = -15$ gives the lower value, -390.3858 versus -175.8626 — a gap of about 214.5. Local optimality with a small gradient certifies only the basin; a *global* claim needs multistart over a dense set of starts, a dense grid or global optimizer as cross-check, enumeration of all stationary points (a proof rather than evidence), and control of the behaviour outside the search window.

```
starts <- seq(-40, 5, by = 0.5)
ms <- t(sapply(starts, function(s) {
  o <- optim(s, f, fp, method = "BFGS")
  c(x = o$par, value = o$value)
}))
table(round(as.data.frame(ms)$x, 4))    # distinct limit points
#>
#> -32.6494 -32.6493 -32.6492 -32.6491  -32.649 -32.6489 -32.6488 -32.6487
#>        5        3        5       48        3        2        1        3
#>   2.3499     2.35   2.3501
#>        1       18        2
min(as.data.frame(ms)$value)
#> [1] -390.386

xw <- seq(-200, 8, length.out = 400001)  # dense-grid cross-check
c(argmin = xw[which.min(f(xw))], min = min(f(xw)))
#>    argmin       min
#>  -32.6489 -390.3858
```

All three agree, and here the argument closes analytically: for $x \lesssim -20$ the term $1.5e^{1.5x}$ is negligible, so $f'(x) = 0$ reduces to $0.15x^2 + 6x + 36 = 0$ with roots $(-40 \pm \sqrt{640})/2 \in \{-7.351,\ -32.649\}$ — exactly the local maximum and left minimum — while for $x > 3$ the exponential makes f' strictly positive. There are therefore exactly three stationary points on $\mathbb{R}$ and the global minimum is the one found from $x^{(0)} = -15$: $x^\star \approx -32.6491$, $f(x^\star) \approx -390.3858$.

Question 4 (Nearly collinear predictors)

Solution

Part a. *(condition numbers and training RMSE for* $\mathbf{X}$ *and* $\mathbf{X}_+$*; coefficients of* $\mathbf{x}_1, \mathbf{x}_6$*.)*

```
set.seed(43202)
u <- rnorm(n)
x6 <- X[, "x1"] + 1e-4 * u
X_plus <- cbind(X, x6 = x6)
y_star <- y + 1e-3 * u

fit_ls <- function(design, response) {
  b <- qr.coef(qr(design), response)
  fitted <- as.vector(design %*% b)
  list(coef = b, fitted = fitted,
       rmse = sqrt(mean((response - fitted)^2)),
       kappa = kappa(design, exact = TRUE))
}

fit_X  <- fit_ls(X, y)
fit_Xp <- fit_ls(X_plus, y)

data.frame(design = c("X", "X_plus"),
           condition_number = c(fit_X$kappa, fit_Xp$kappa),
           train_RMSE = c(fit_X$rmse, fit_Xp$rmse))
#>   design condition_number train_RMSE
#> 1      X          1.52491   0.676370
#> 2 X_plus      20209.20350   0.675467

c(beta_x1 = fit_Xp$coef[["x1"]], beta_x6 = fit_Xp$coef[["x6"]],
  sum = fit_Xp$coef[["x1"]] + fit_Xp$coef[["x6"]])
#>    beta_x1    beta_x6        sum
#>  351.67661 -350.25876    1.41785
```

The condition number jumps from $\kappa(\mathbf{X}) = 1.525$ — essentially perfect for independent standard normal columns — to $\kappa(\mathbf{X}_+) = 20,209$, four orders of magnitude larger, because adding $\mathbf{x}_6$ creates a direction along which $\mathbf{X}_+$ has almost no length: $\sigma_{\min}(\mathbf{X}_+) = 0.000705$ against 7.48. The matrix is still formally of full rank 7, so the fit is unique, but numerically it is almost rank deficient.

Training RMSE barely moves (0.67637 versus 0.67547), since $\mathbf{x}_6$ carries almost no information not already in $\mathbf{x}_1$. The individual coefficients, by contrast, are wild — 351.68 and −350.26, hundreds of times any true coefficient and of opposite sign — yet their **sum**, 1.4178, is almost exactly the Question 1 estimate 1.4172. Only the sum is well determined; the split is set by the tiny $10^{-4}\mathbf{u}$ and is arbitrary.

Part b. *(refit on* $\mathbf{y}^\star$*: coefficient changes and fitted-value differences.)*

```
fit_Xp_star <- fit_ls(X_plus, y_star)

coef_cmp <- data.frame(
  fit_y = c(fit_Xp$coef[["x1"]], fit_Xp$coef[["x6"]]),
  fit_y_star = c(fit_Xp_star$coef[["x1"]], fit_Xp_star$coef[["x6"]]),
  change = c(fit_Xp_star$coef[["x1"]] - fit_Xp$coef[["x1"]],
             fit_Xp_star$coef[["x6"]] - fit_Xp$coef[["x6"]])
)
rownames(coef_cmp) <- c("x1", "x6")
round(coef_cmp, 6)
#>       fit_y fit_y_star change
#> x1  351.677    341.677    -10
#> x6 -350.259   -340.259     10
```

```r
d_fit <- fit_Xp_star$fitted - fit_Xp$fitted
c(rms_difference = sqrt(mean(d_fit^2)),
  max_abs_difference = max(abs(d_fit)),
  sum_before = fit_Xp$coef[["x1"]] + fit_Xp$coef[["x6"]],
  sum_after = fit_Xp_star$coef[["x1"]] + fit_Xp_star$coef[["x6"]])
#>     rms_difference max_abs_difference         sum_before
#>         0.00101588         0.00257268         1.41785000
#>          sum_after
#>         1.41785000
```

Perturbing the response by $10^{-3}\mathbf{u}$ — about 0.045% of sd($\mathbf{y}$) — changes $\hat{\beta}_{x_1}$ by exactly -10 and $\hat{\beta}_{x_6}$ by exactly 10, leaving their sum unchanged to machine precision, while the fitted values move by only 0.001016 (RMS) and 0.002573 (max) — an amplification of five orders of magnitude, exactly what $\kappa \approx 20,209$ warns about.

Part c. *(the identity $\mathbf{y}^\star - \mathbf{y} = 10(\mathbf{x}_6 - \mathbf{x}_1)$, and interpreting collinear predictors.)* Since $\mathbf{x}_6 - \mathbf{x}_1 = 10^{-4}\mathbf{u}$,

$$\mathbf{y}^\star - \mathbf{y} = 10^{-3}\mathbf{u} = 10\,(\mathbf{x}_6 - \mathbf{x}_1)\,,$$

so the perturbation lies **exactly in the column space of $\mathbf{X}_+$** — it is $\mathbf{X}_+\delta$ with $\delta = 10(\mathbf{e}_{x_6} - \mathbf{e}_{x_1})$. Least squares is linear, so with $\mathbf{H}_+$ the projection onto $\mathcal{C}(\mathbf{X}_+)$,

$$\hat{\beta}^\star = \hat{\beta} + \delta, \qquad \hat{\mathbf{y}}^\star - \hat{\mathbf{y}} = \mathbf{H}_+\,(\mathbf{y}^\star - \mathbf{y}) = \mathbf{y}^\star - \mathbf{y} = 10^{-3}\mathbf{u}.$$

This predicts both observations exactly: coefficients change by -10 on $\mathbf{x}_1$ and $+10$ on $\mathbf{x}_6$ and by zero elsewhere, leaving the sum invariant, and fitted values change by exactly $10^{-3}\mathbf{u}$ — the part b numbers.

```r
max(abs((y_star - y) - 10 * (x6 - X[, "x1"])))    # the identity
#> [1] 1.77636e-15
max(abs(d_fit - 1e-3 * u))                        # fitted change is exactly 1e-3 * u
#> [1] 1.00724e-13
```

The mechanism is a near-null direction: $\mathbf{v} = \mathbf{e}_{x_6} - \mathbf{e}_{x_1}$ has $\|\mathbf{X}_+\mathbf{v}\| = 0.00102$, essentially zero, so RSS is nearly flat along $\mathbf{v}$ — a long horizontal trough rather than a well-defined bowl — and the position along it is set by whatever minuscule signal is present. Formally $\sigma^2(\mathbf{X}_+^\mathsf{T}\mathbf{X}_+)^{-1}$ has an eigenvalue of order $\sigma^2 \cdot 10^6$ along $\mathbf{v}$, so the two coefficients have enormous variance and are almost perfectly negatively correlated, while their sum — orthogonal to $\mathbf{v}$ — is as precisely estimated as β_{x_1} in Q1.

On interpretation. With nearly collinear predictors the data cannot separate individual effects, though they pin down the combined effect and the predictions well. Prediction is fine; individual coefficients, their signs, their standard errors and any "holding the other fixed" story are not interpretable — "a one-unit increase in x_1 raises y by 352, holding x_6 fixed" is meaningless, because no observation has x_1 moving while x_6 stays put. What is estimable is $\beta_{x_1} + \beta_{x_6}$, and that is what should be reported, with a diagnostic such as the condition number or VIF. If separate effects are genuinely needed the fix must come from outside the data: drop or combine the redundant variables, break the collinearity with new data, or regularize.

Question 5 (Data preparation and summary statistics)

Solution

Part a. *(dimensions, types, duplicates and missingness; the analysis table and `feature_sum`.)*

```r
dat <- read.csv("data/data-manipulation.csv", stringsAsFactors = FALSE)

dim(dat)
#> [1] 24  5
sapply(dat, class)
#> observation_id          group             x1             x2
#>    "character"    "character"      "numeric"      "numeric"
#>       response
```

```
#>      "numeric"
n_dup_ids <- sum(duplicated(dat$observation_id))
n_dup_ids
#> [1] 0
colSums(is.na(dat))
#> observation_id          group             x1             x2
#>              0              0              0              0
#>       response
#>              3
```

The file has 24 rows and 5 columns; `observation_id` and `group` are `character`, `x1`, `x2` and `response` are `numeric`. There are 0 duplicated identifiers, and the only missing values are 3 responses (obs-04, obs-11, obs-20). As instructed those rows are **dropped for response-based analyses** rather than imputed: a zero would fabricate an extreme low value and drag group means down, and a group mean would falsely shrink the variance and overstate the effective sample size. Complete-case analysis is unbiased here provided missingness does not depend on the unobserved response.

```
analysis <- dat[!is.na(dat$response), ]
analysis$feature_sum <- analysis$x1 + analysis$x2
rownames(analysis) <- NULL

dim(analysis)
#> [1] 21  6
head(analysis, 3)
#>   observation_id group  x1  x2 response feature_sum
#> 1         obs-01     A 1.2 2.0     5.36         3.2
#> 2         obs-02     A 2.4 1.1     6.18         3.5
#> 3         obs-03     A 0.8 2.8     4.62         3.6
```

Part b. *(per-group n, mean response and mean `feature_sum`.)*

```
group_summary <- do.call(rbind, lapply(
  split(analysis, analysis$group),
  function(d) data.frame(group = d$group[1], n = nrow(d),
                         mean_response = mean(d$response),
                         mean_feature_sum = mean(d$feature_sum))
))
rownames(group_summary) <- NULL
group_summary
#>   group n mean_response mean_feature_sum
#> 1     A 7       5.60000          3.60000
#> 2     B 7       7.60286          4.92857
#> 3     C 7       7.99714          6.58571

# rows each group lost to a missing response
rbind(raw = table(dat$group), analysis = table(analysis$group))
#>          A B C
#> raw      8 8 8
#> analysis 7 7 7
```

These summaries describe only the observations with a recorded response — the 21 rows of the analysis table, not all 24 raw rows. Each group began with 8 observations and lost exactly one, so every summary uses $n = 7$; because the loss is balanced, the comparison is not distorted by differential attrition. Both summaries are monotone in the group label — mean response rises from 5.6 (A) to 7.603 (B) to 7.997 (C), and mean `feature_sum` in the same order — so the groups differ in predictor levels as well as in response, and a raw comparison confounds the label with `feature_sum`.

Part c. *(three largest responses, plus the verification checks.)*

```r
sorted <- analysis[order(analysis$response, decreasing = TRUE), ]
top3 <- sorted[1:3, c("observation_id", "group", "response", "feature_sum")]
rownames(top3) <- NULL
top3
#>   observation_id group response feature_sum
#> 1         obs-22     C     9.24         6.6
#> 2         obs-16     B     9.00         5.2
#> 3         obs-14     B     8.66         5.0
```

```r
check_row_count <- nrow(analysis) == sum(!is.na(dat$response))
check_no_missing <- sum(is.na(analysis$response)) == 0
check_unique_ids <- !any(duplicated(analysis$observation_id))

c("retained rows == nonmissing responses" = check_row_count,
  "no missing response in analysis table" = check_no_missing,
  "identifiers are unique"                = check_unique_ids)
#> retained rows == nonmissing responses
#>                                  TRUE
#> no missing response in analysis table
#>                                  TRUE
#>                identifiers are unique
#>                                  TRUE

# hard stops: the render fails if any check is violated
stopifnot(check_row_count, check_no_missing, check_unique_ids)
```

All three checks pass. `stopifnot()` is used rather than a printed message so the document cannot render if the data-preparation step is ever broken upstream.

Question 6 (Can 39 million Fitbit records represent US adults?)

Solution

Part a. *(independence of the 39 million records, and unequal weighting of participants.)* No. The records are repeated measurements clustered within only 59,018 people — roughly 660 days each. Days from the same person are strongly positively correlated, since activity level, commute, occupation, health and neighbourhood walkability persist across days, with further serial dependence within a person (weekday cycles, seasonality, illness) and shared shocks across people (weather, holidays, COVID-19 inside a 14-year window). The effective sample size for a person-level target is therefore of the order of the number of *participants*, not records: treating $N = 39$ million as the sample size understates the standard error by about $\sqrt{1 + (\bar{m} - 1)\rho}$ with $\bar{m} \approx 660$ — a factor of about 14 even at $\rho = 0.3$.

Days per person is itself an outcome, driven by length of enrollment and calendar timing; device ownership and upgrades; wear adherence and charging habits, themselves activity-related; motivation and engagement; health and life circumstances, since illness or a broken device create gaps; and processing rules filtering out days with too few wear hours. If participant i contributes m_i days with mean $\bar{s}_i$, the average of all records is *record-weighted*,

$$\hat{\mu}_{\text{naive}} = \frac{\sum_i \sum_{j=1}^{m_i} s_{ij}}{\sum_i m_i} = \sum_i w_i \, \bar{s}_i, \qquad w_i = \frac{m_i}{\sum_k m_k},$$

whereas the target weights adults equally, $\mu = \frac{1}{N} \sum_i \bar{s}_i$. These agree only if m_i is constant or uncorrelated with $\bar{s}_i$ — but m_i plausibly rises with adherence, engagement and good health, the same traits associated with more steps. The gap is exactly $\text{Cov}_n(m_i, \bar{s}_i)/\bar{m}$; averaging each person's own mean first removes it.

Part b. *(whether the BYOD/WEAR gap is causal.)* No. The groups were formed by completely different, non-random processes, so the comparison is observational and confounded by construction. BYOD participants **self-selected**: they already owned a Fitbit and chose to share its data. Buying a tracker and continuing to wear it marks an existing interest in exercise, higher disposable income and health consciousness — all of which

independently produce more walking — whereas WEAR participants were *invited* and given a device free, so that group includes many who would never have bought one. The reported numbers confirm the groups differ sharply on characteristics associated with activity: 77.3% versus 55.1% reported being White, and 6.1% versus 15.2% fell in the \$10,000–\$25,000 income band, making WEAR participants 2.5 times as likely to be low-income. Socioeconomic status is a well-documented correlate of measured activity through neighbourhood walkability, occupation, work schedules and chronic disease burden — a plausible **common cause** of both owning a Fitbit and walking more, so the 1,070-step gap is at least partly, and possibly entirely, confounding.

Other explanations rooted in how people entered the groups: **reverse causation**, where people who already walk a lot are the ones who buy a tracker; **differential measurement and adherence**, since experienced owners wear the device during active parts of the day while new recipients may wear a novel device inconsistently; **different calendar coverage** around 2020; and **different device generations** with different step algorithms. A causal claim would need randomized device provision, or adjustment for measured confounders plus an argument for no important unmeasured ones.

Part c. *(selection of participants and of recorded days; why the naive average fails.)* Size does not fix selection: increasing n shrinks *variance*, but bias from a selected sample does not shrink at all — a larger biased sample estimates the wrong quantity more precisely. In Meng's decomposition,

$$\hat{\mu}_{\text{obs}} - \mu \;\approx\; \underbrace{\rho_{R,S}}_{\text{data defect}} \;\times\; \underbrace{\sqrt{\frac{1-f}{f}}}_{\text{data quantity}} \;\times\; \underbrace{\sigma_S}_{\text{problem difficulty}} \,,$$

where R indicates inclusion, S is the step count and f the sampling fraction. When inclusion correlates with the outcome the error is essentially independent of sample size except through f, so 39 million records with a tiny defect correlation can be worse than a small probability sample.

Selection operates at three nested levels. *Participants into the study*: All of Us is a volunteer cohort requiring awareness of the program, willingness to consent to long-term data sharing and typically proximity to a participating site, with the analysis further restricted to those contributing Fitbit data at all — so no US adult has a known, positive, quantifiable probability of appearing, and 77.3% White in BYOD is far above the US adult share. *Participants into the two routes*, as in part b, makes the data a mixture of two very different populations in proportions reflecting program logistics. *The recorded days*: a day appears only if the device was worn, charged, synced and passed wear-time filters, so sedentary days, sick days and hospitalizations are systematically more likely to be missing while deliberate-exercise days are more likely to be recorded.

The naive average therefore equals $\sum_{i \in \text{cohort}} \frac{m_i}{\sum_k m_k} \bar{s}_i^{\text{obs}}$ and departs from μ in three compounding ways: the **wrong population**, since the sum runs over volunteers with wearables with no reweighting; the **wrong weights**, since participants are weighted by m_i rather than equally, incurring $\text{Cov}(m_i, \bar{s}_i)/\bar{m}$; and the **wrong per-person quantity**, since $\bar{s}_i^{\text{obs}}$ averages only observed days. These are biases, not variance, so they do not vanish as records accumulate: an interval built from $N = 39$ million would be vanishingly narrow and centered in the wrong place. Partial remedies are to average within person first, reweight to US adult margins using benchmarks such as NHANES or the ACS, impose a common wear-time standard while modelling the missing days, analyze BYOD and WEAR separately, and otherwise report the result as descriptive of this cohort rather than of US adults.